

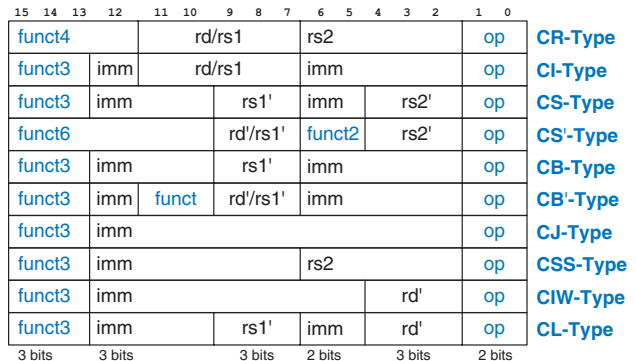
Digital Design and Computer Architecture

RISC-V Edition



Table B.4 Register names and numbers

Name	Register Number	Use
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0–2	x5–7	Temporary registers
s0/fp	x8	Saved register / Frame pointer
s1	x9	Saved register
a0–1	x10–11	Function arguments / Return values
a2–7	x12–17	Function arguments
s2–11	x18–27	Saved registers
t3–6	x28–31	Temporary registers

**Figure B.2 RISC-V compressed (16-bit) instruction formats****Table B.5 RVM: RISC-V multiply and divide instructions**

op	funct3	funct7	Type	Instruction	Description	Operation
0110011 (51)	000	0000001	R	mul rd, rs1, rs2	multiply	$rd = (rs1 * rs2)_{31:0}$
0110011 (51)	001	0000001	R	mulh rd, rs1, rs2	multiply high signed signed	$rd = (rs1 * rs2)_{63:32}$
0110011 (51)	010	0000001	R	mulhsu rd, rs1, rs2	multiply high signed unsigned	$rd = (rs1 * rs2)_{63:32}$
0110011 (51)	011	0000001	R	mulhu rd, rs1, rs2	multiply high unsigned unsigned	$rd = (rs1 * rs2)_{63:32}$
0110011 (51)	100	0000001	R	div rd, rs1, rs2	divide (signed)	$rd = rs1 / rs2$
0110011 (51)	101	0000001	R	divu rd, rs1, rs2	divide unsigned	$rd = rs1 / rs2$
0110011 (51)	110	0000001	R	rem rd, rs1, rs2	remainder (signed)	$rd = rs1 \% rs2$
0110011 (51)	111	0000001	R	remu rd, rs1, rs2	remainder unsigned	$rd = rs1 \% rs2$

Table B.6 RVC: RISC-V compressed (16-bit) instructions

op	instr _{15:10}	funct2	Type	RVC Instruction	32-Bit Equivalent
00 (0)	000---	—	CIW	c.addi4spn rd', sp, imm	addi rd', sp, ZeroExt(imm)*4
00 (0)	001---	—	CL	c.fld fd', imm(rs1')	fld fd', (ZeroExt(imm)*8)(rs1')
00 (0)	010---	—	CL	c.lw rd', imm(rs1')	lw rd', (ZeroExt(imm)*4)(rs1')
00 (0)	011---	—	CL	c.flw fd', imm(rs1')	flw fd', (ZeroExt(imm)*4)(rs1')
00 (0)	101---	—	CS	c.fsd fs2', imm(rs1')	fsd fs2', (ZeroExt(imm)*8)(rs1')
00 (0)	110---	—	CS	c.sw rs2', imm(rs1')	sw rs2', (ZeroExt(imm)*4)(rs1')
00 (0)	111---	—	CS	c.fsw fs2', imm(rs1')	fsw fs2', (ZeroExt(imm)*4)(rs1')
01 (1)	000000	—	CI	c.nop (rs1=0,imm=0)	nop
01 (1)	000---	—	CI	c.addi rd, imm	addi rd, rd, SignExt(imm)
01 (1)	001---	—	CJ	c.jal label	jal ra, label
01 (1)	010---	—	CI	c.li rd, imm	addi rd, x0, SignExt(imm)
01 (1)	011---	—	CI	c.lui rd, imm	lui rd, {14{imm ₅ }, imm}
01 (1)	011---	—	CI	c.addi16sp sp, imm	addi sp, sp, SignExt(imm)*16
01 (1)	100--0	—	CB'	c.srli rd', imm	srlr rd', rd', imm
01 (1)	100-01	—	CB'	c.srai rd', imm	srair rd', rd', imm
01 (1)	100-10	—	CB'	c.andi rd', imm	andi rd', rd', SignExt(imm)
01 (1)	100011	00	CS'	c.sub rd', rs2'	sub rd', rd', rs2'
01 (1)	100011	01	CS'	c.xor rd', rs2'	xor rd', rd', rs2'
01 (1)	100011	10	CS'	c.or rd', rs2'	or rd', rd', rs2'
01 (1)	100011	11	CS'	c.and rd', rs2'	and rd', rd', rs2'
01 (1)	101---	—	CJ	c.j label	jal x0, label
01 (1)	110---	—	CB	c.beqz rs1', label	beq rs1', x0, label
01 (1)	111---	—	CB	c.bnez rs1', label	bne rs1', x0, label
10 (2)	000---	—	CI	c.slli rd, imm	sllr rd, rd, imm
10 (2)	001---	—	CI	c.fldsp fd, imm	fld fd, (ZeroExt(imm)*8)(sp)
10 (2)	010---	—	CI	c.lwsp rd, imm	lw rd, (ZeroExt(imm)*4)(sp)
10 (2)	011---	—	CI	c.flwsp fd, imm	flw fd, (ZeroExt(imm)*4)(sp)
10 (2)	1000--	—	CR	c.jr rs1 (rs1≠0,rs2=0)	jalr x0, rs1, 0
10 (2)	1000--	—	CR	c.mv rd, rs2 (rd≠0,rs2≠0)	add rd, x0, rs2
10 (2)	1001--	—	CR	c.ebreak (rs1=0,rs2=0)	ebreak
10 (2)	1001--	—	CR	c.jalr rs1 (rs1≠0,rs2=0)	jalr ra, rs1, 0
10 (2)	1001--	—	CR	c.add rd, rs2 (rs1≠0,rs2≠0)	add rd, rd, rs2
10 (2)	101---	—	CSS	c.fsdsp fs2, imm	fsd fs2, (ZeroExt(imm)*8)(sp)
10 (2)	110---	—	CSS	c.swsp rs2, imm	sw rs2, (ZeroExt(imm)*4)(sp)
10 (2)	111---	—	CSS	c.fswsp fs2, imm	fsw fs2, (ZeroExt(imm)*4)(sp)

rs1', rs2', rd': 3-bit register designator for registers 8–15: 000₂ = x8 or f8, 001₂ = x9 or f9, etc.

Table B.7 RISC-V pseudoinstructions

Pseudoinstruction	RISC-V Instructions	Description	Operation
nop	addi x0, x0, 0	no operation	
li rd, imm _{11:0}	addi rd, x0, imm _{11:0}	load 12-bit immediate	rd = SignExtend(imm _{11:0})
li rd, imm _{31:0}	lui rd, imm _{31:12} addi rd, rd, imm _{11:0}	load 32-bit immediate	rd = imm _{31:0}
mv rd, rs1	addi rd, rs1, 0	move (also called “register copy”)	rd = rs1
not rd, rs1	xori rd, rs1, -1	one’s complement	rd = ~rs1
neg rd, rs1	sub rd, x0, rs1	two’s complement	rd = -rs1
seqz rd, rs1	sltiu rd, rs1, 1	set if = 0	rd = (rs1 == 0)
snez rd, rs1	sltu rd, x0, rs1	set if ≠ 0	rd = (rs1 ≠ 0)
sltz rd, rs1	slt rd, rs1, x0	set if < 0	rd = (rs1 < 0)
sgtz rd, rs1	slt rd, x0, rs1	set if > 0	rd = (rs1 > 0)
beqz rs1, label	beq rs1, x0, label	branch if = 0	if (rs1 == 0) PC = label
bnez rs1, label	bne rs1, x0, label	branch if ≠ 0	if (rs1 ≠ 0) PC = label
blez rs1, label	bge x0, rs1, label	branch if ≤ 0	if (rs1 ≤ 0) PC = label
bgez rs1, label	bge rs1, x0, label	branch if ≥ 0	if (rs1 ≥ 0) PC = label
bltz rs1, label	blt rs1, x0, label	branch if < 0	if (rs1 < 0) PC = label
bgtz rs1, label	blt x0, rs1, label	branch if > 0	if (rs1 > 0) PC = label
ble rs1, rs2, label	bge rs2, rs1, label	branch if ≤	if (rs1 ≤ rs2) PC = label
bgt rs1, rs2, label	blt rs2, rs1, label	branch if >	if (rs1 > rs2) PC = label
bleu rs1, rs2, label	bgeu rs2, rs1, label	branch if ≤ (unsigned)	if (rs1 ≤ rs2) PC = label
bgtu rs1, rs2, label	bltu rs2, rs1, offset	branch if > (unsigned)	if (rs1 > rs2) PC = label
j label	jal x0, label	jump	PC = label
jal label	jal ra, label	jump and link	PC = label, ra = PC + 4
jr rs1	jalr x0, rs1, 0	jump register	PC = rs1
jalr rs1	jalr ra, rs1, 0	jump and link register	PC = rs1, ra = PC + 4
ret	jalr x0, ra, 0	return from function	PC = ra
call label	jal ra, label	call nearby function	PC = label, ra = PC + 4
call label	auipc ra, offset _{31:12} jalr ra, ra, offset _{11:0}	call far away function	PC = PC + offset, ra = PC + 4
la rd, symbol	auipc rd, symbol _{31:12} addi rd, rd, symbol _{11:0}	load address of global variable	rd = PC + symbol
l{b h w} rd, symbol	auipc rd, symbol _{31:12} l{b h w} rd, symbol _{11:0} (rd)	load global variable	rd = [PC + symbol]
s{b h w} rs2, symbol, rs1	auipc rs1, symbol _{31:12} s{b h w} rs2, symbol _{11:0} (rs1)	store global variable	[PC + symbol] = rs2
crr rd, csr	crrs rd, csr, x0	read CSR	rd = csr
crrw csr, rs1	crrw x0, csr, rs1	write CSR	csr = rs1

* If bit 11 of the immediate / offset / symbol is 1, the upper immediate is incremented by 1. symbol and offset are the 32-bit PC-relative addresses of a label and a global variable, respectively.

Table B.8 Privileged / CSR instructions

op	funct3	Type	Instruction	Description	Operation
1110011 (115)	000	I	ecall	transfer control to OS (imm=0)	
1110011 (115)	000	I	ebreak	transfer control to debugger (imm=1)	
1110011 (115)	000	I	uret	return from user exception (rs1=0,rd=0,imm=2)	PC = uepc
1110011 (115)	000	I	sret	return from supervisor exception (rs1=0,rd=0,imm=258)	PC = sepc
1110011 (115)	000	I	mret	return from machine exception (rs1=0,rd=0,imm=770)	PC = mepc
1110011 (115)	001	I	crrw rd,csr,rs1	CSR read/write (imm=CSR number)	rd = csr, csr = rs1
1110011 (115)	010	I	crrs rd,csr,rs1	CSR read/set (imm=CSR number)	rd = csr, csr = csr rs1
1110011 (115)	011	I	crrc rd,csr,rs1	CSR read/clear (imm=CSR number)	rd = csr, csr = csr & ~rs1
1110011 (115)	101	I	crrwi rd,csr,uimm	CSR read/write immediate (imm=CSR number)	rd = csr, csr = ZeroExt(uimm)
1110011 (115)	110	I	crrsi rd,csr,uimm	CSR read/set immediate (imm=CSR number)	rd = csr, csr = csr ZeroExt(uimm)
1110011 (115)	111	I	crrci rd,csr,uimm	CSR read/clear immediate (imm=CSR number)	rd = csr, csr = csr & ~ZeroExt(uimm)

For privileged / CSR instructions, the 5-bit unsigned immediate, uimm, is encoded in the rs1 field.

In Praise of *Digital Design and Computer Architecture*

RISC-V Edition

Harris and Harris have shown how to design a RISC-V processor from the gates all the way up through the microarchitecture. Their clear explanations combined with their comprehensive approach give a full picture of both digital design and the RISC-V architecture. With the exciting opportunity that students have to run large digital designs on modern FPGAs, this approach is both informative and enlightening.

David A. Patterson University of California, Berkeley

What broad fields Harris and Harris have distilled into one book! As semiconductor manufacturing matures, the importance of digital design and computer architecture will only increase. Readers will find an approachable and comprehensive treatment of both topics and will walk away with a clear understanding of the RISC-V instruction set architecture.

Andrew Waterman SiFive

There are excellent texts for teaching digital design and there are excellent texts for teaching computer hardware organization - and this textbook does both! It is also unique in its ability to connect the dots. The writing makes the RISC-V architecture understandable by building on the basics, and I have found the exercises to be a great resource across multiple courses.

Roy Kravitz Portland State University

When I first read Harris and Harris's MIPS textbook back in 2008, I thought that it was one of the best books I had ever read for teaching computer architecture. I started using it in my courses immediately. Thirteen years later, I have had the honor of reviewing this new RISC-V edition, and my opinion of their books has not changed: *Digital Design and Computer Architecture: RISC-V Edition* is an excellent book, very clear, thorough, with a high educational value, and in line with the courses that we teach in the areas of digital design and computer architecture. I look forward to using this RISC-V textbook in my courses.

Daniel Chaver Martinez University Complutense of Madrid

Digital Design and Computer Architecture

RISC-V Edition

Digital Design and Computer Architecture

RISC-V Edition

Sarah L. Harris
David Harris



ELSEVIER

AMSTERDAM • BOSTON • HEIDELBERG • LONDON
NEW YORK • OXFORD • PARIS • SAN DIEGO
SAN FRANCISCO • SINGAPORE • SYDNEY • TOKYO

Morgan Kaufmann is an imprint of Elsevier

MK
MORGAN KAUFMANN

Morgan Kaufmann is an imprint of Elsevier
50 Hampshire Street, 5th Floor, Cambridge, MA 02139, United States

Copyright © 2022 Elsevier Inc. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or any information storage and retrieval system, without permission in writing from the publisher. Details on how to seek permission, further information about the Publisher's permissions policies and our arrangements with organizations such as the Copyright Clearance Center and the Copyright Licensing Agency, can be found at our website: www.elsevier.com/permissions.

This book and the individual contributions contained in it are protected under copyright by the Publisher (other than as may be noted herein).

Notices

Knowledge and best practice in this field are constantly changing. As new research and experience broaden our understanding, changes in research methods, professional practices, or medical treatment may become necessary.

Practitioners and researchers must always rely on their own experience and knowledge in evaluating and using any information, methods, compounds, or experiments described herein. In using such information or methods they should be mindful of their own safety and the safety of others, including parties for whom they have a professional responsibility.

To the fullest extent of the law, neither the Publisher nor the authors, contributors, or editors, assume any liability for any injury and/or damage to persons or property as a matter of products liability, negligence or otherwise, or from any use or operation of any methods, products, instructions, or ideas contained in the material herein.

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library.

Library of Congress Cataloging-in-Publication Data

A catalog record for this book is available from the Library of Congress.

ISBN: 978-0-12-820064-3

For Information on all Morgan Kaufmann publications visit our website at
<https://www.elsevier.com/books-and-journals>

Publisher: Katey Birtcher

Senior Acquisitions Editor: Steve Merken

Editorial Project Manager: Ruby Gammell/Andrae Akeh

Senior Project Manager: Manikandan Chandrasekaran

Cover Designer: Victoria Pearson Esser

Typeset by MPS Limited, Chennai, India

Printed in the United States of America

Last digit is the print number: 9 8 7 6 5 4 3 2 1



Working together
to grow libraries in
developing countries

www.elsevier.com • www.bookaid.org

Contents

Preface	xix
Features	xx
Online Supplements	xxi
How to Use the Software Tools in a Course.....	xxii
Labs	xxiii
RVfpga.....	xxiii
Bugs	xxiv
Acknowledgments.....	xxiv
About the Authors	xxv
Chapter 1 From Zero to One	1
1.1 The Game Plan	1
1.2 The Art of Managing Complexity.....	2
1.2.1 <i>Abstraction</i>	2
1.2.2 <i>Discipline</i>	3
1.2.3 <i>The Three -Y's</i>	4
1.3 The Digital Abstraction.....	5
1.4 Number Systems	7
1.4.1 <i>Decimal Numbers</i>	7
1.4.2 <i>Binary Numbers</i>	7
1.4.3 <i>Hexadecimal Numbers</i>	9
1.4.4 <i>Bytes, Nibbles, and All That Jazz</i>	11
1.4.5 <i>Binary Addition</i>	12
1.4.6 <i>Signed Binary Numbers</i>	13
1.5 Logic Gates.....	17
1.5.1 <i>NOT Gate</i>	18
1.5.2 <i>Buffer</i>	18
1.5.3 <i>AND Gate</i>	18
1.5.4 <i>OR Gate</i>	19
1.5.5 <i>Other Two-Input Gates</i>	19
1.5.6 <i>Multiple-Input Gates</i>	19
1.6 Beneath the Digital Abstraction.....	20
1.6.1 <i>Supply Voltage</i>	20
1.6.2 <i>Logic Levels</i>	20
1.6.3 <i>Noise Margins</i>	21

1.6.4	DC Transfer Characteristics	22
1.6.5	The Static Discipline	22
1.7	CMOS Transistors	24
1.7.1	Semiconductors	25
1.7.2	Diodes	25
1.7.3	Capacitors	26
1.7.4	nMOS and pMOS Transistors	26
1.7.5	CMOS NOT Gate	29
1.7.6	Other CMOS Logic Gates	29
1.7.7	Transmission Gates	31
1.7.8	Pseudo-nMOS Logic	31
1.8	Power Consumption	32
1.9	Summary and a Look Ahead	34
	Exercises	36
	Interview Questions	50
	Chapter 2 Combinational Logic Design	53
2.1	Introduction	53
2.2	Boolean Equations	56
2.2.1	Terminology	56
2.2.2	Sum-of-Products Form	56
2.2.3	Product-of-Sums Form	58
2.3	Boolean Algebra	58
2.3.1	Axioms	59
2.3.2	Theorems of One Variable	59
2.3.3	Theorems of Several Variables	60
2.3.4	The Truth Behind It All	62
2.3.5	Simplifying Equations	63
2.4	From Logic to Gates	64
2.5	Multilevel Combinational Logic	67
2.5.1	Hardware Reduction	68
2.5.2	Bubble Pushing	69
2.6	X's and Z's, Oh My	71
2.6.1	Illegal Value: X	71
2.6.2	Floating Value: Z	72
2.7	Karnaugh Maps	73
2.7.1	Circular Thinking	74
2.7.2	Logic Minimization with K-Maps	75
2.7.3	Don't Cares	79
2.7.4	The Big Picture	80
2.8	Combinational Building Blocks	81
2.8.1	Multiplexers	81
2.8.2	Decoders	84

2.9	Timing	86
2.9.1	<i>Propagation and Contamination Delay</i>	86
2.9.2	<i>Glitches</i>	90
2.10	Summary	93
	Exercises	95
	Interview Questions	104
Chapter 3 Sequential Logic Design		107
3.1	Introduction	107
3.2	Latches and Flip-Flops	107
3.2.1	<i>SR Latch</i>	109
3.2.2	<i>D Latch</i>	111
3.2.3	<i>D Flip-Flop</i>	112
3.2.4	<i>Register</i>	112
3.2.5	<i>Enabled Flip-Flop</i>	113
3.2.6	<i>Resettable Flip-Flop</i>	114
3.2.7	<i>Transistor-Level Latch and Flip-Flop Designs</i>	114
3.2.8	<i>Putting It All Together</i>	116
3.3	Synchronous Logic Design	117
3.3.1	<i>Some Problematic Circuits</i>	117
3.3.2	<i>Synchronous Sequential Circuits</i>	118
3.3.3	<i>Synchronous and Asynchronous Circuits</i>	120
3.4	Finite State Machines	121
3.4.1	<i>FSM Design Example</i>	121
3.4.2	<i>State Encodings</i>	127
3.4.3	<i>Moore and Mealy Machines</i>	130
3.4.4	<i>Factoring State Machines</i>	132
3.4.5	<i>Deriving an FSM from a Schematic</i>	135
3.4.6	<i>FSM Review</i>	138
3.5	Timing of Sequential Logic	139
3.5.1	<i>The Dynamic Discipline</i>	140
3.5.2	<i>System Timing</i>	140
3.5.3	<i>Clock Skew</i>	146
3.5.4	<i>Metastability</i>	149
3.5.5	<i>Synchronizers</i>	150
3.5.6	<i>Derivation of Resolution Time</i>	152
3.6	Parallelism	155
3.7	Summary	159
	Exercises	160
	Interview Questions	169

Chapter 4	Hardware Description Languages	171
4.1	Introduction	171
4.1.1	Modules	171
4.1.2	Language Origins	172
4.1.3	Simulation and Synthesis	173
4.2	Combinational Logic	175
4.2.1	Bitwise Operators	175
4.2.2	Comments and White Space	178
4.2.3	Reduction Operators	178
4.2.4	Conditional Assignment	179
4.2.5	Internal Variables	180
4.2.6	Precedence	182
4.2.7	Numbers	183
4.2.8	Z's and X's	184
4.2.9	Bit Swizzling	186
4.2.10	Delays	186
4.3	Structural Modeling	188
4.4	Sequential Logic	191
4.4.1	Registers	191
4.4.2	Resettable Registers	192
4.4.3	Enabled Registers	194
4.4.4	Multiple Registers	195
4.4.5	Latches	196
4.5	More Combinational Logic	196
4.5.1	Case Statements	199
4.5.2	If Statements	200
4.5.3	Truth Tables with Don't Cares	203
4.5.4	Blocking and Nonblocking Assignments	203
4.6	Finite State Machines	207
4.7	Data Types	211
4.7.1	SystemVerilog	212
4.7.2	VHDL	213
4.8	Parameterized Modules	215
4.9	Testbenches	218
4.10	Summary	222
	Exercises	224
	Interview Questions	235
Chapter 5	Digital Building Blocks	237
5.1	Introduction	237
5.2	Arithmetic Circuits	237
5.2.1	Addition	237
5.2.2	Subtraction	244

5.2.3	<i>Comparators</i>	245
5.2.4	<i>ALU</i>	247
5.2.5	<i>Shifters and Rotators</i>	251
5.2.6	<i>Multiplication</i>	253
5.2.7	<i>Division</i>	254
5.2.8	<i>Further Reading</i>	255
5.3	<i>Number Systems</i>	256
5.3.1	<i>Fixed-Point Number Systems</i>	256
5.3.2	<i>Floating-Point Number Systems</i>	257
5.4	<i>Sequential Building Blocks</i>	261
5.4.1	<i>Counters</i>	261
5.4.2	<i>Shift Registers</i>	262
5.5	<i>Memory Arrays</i>	265
5.5.1	<i>Overview</i>	265
5.5.2	<i>Dynamic Random Access Memory (DRAM)</i>	267
5.5.3	<i>Static Random Access Memory (SRAM)</i>	268
5.5.4	<i>Area and Delay</i>	268
5.5.5	<i>Register Files</i>	269
5.5.6	<i>Read Only Memory (ROM)</i>	269
5.5.7	<i>Logic Using Memory Arrays</i>	271
5.5.8	<i>Memory HDL</i>	272
5.6	<i>Logic Arrays</i>	272
5.6.1	<i>Programmable Logic Array (PLA)</i>	275
5.6.2	<i>Field Programmable Gate Array (FPGA)</i>	276
5.6.3	<i>Array Implementations</i>	282
5.7	<i>Summary</i>	283
	<i>Exercises</i>	285
	<i>Interview Questions</i>	297
Chapter 6 Architecture		299
6.1	<i>Introduction</i>	299
6.2	<i>Assembly Language</i>	300
6.2.1	<i>Instructions</i>	301
6.2.2	<i>Operands: Registers, Memory, and Constants</i>	302
6.3	<i>Programming</i>	308
6.3.1	<i>Program Flow</i>	308
6.3.2	<i>Logical, Shift, and Multiply Instructions</i>	308
6.3.3	<i>Branching</i>	311
6.3.4	<i>Conditional Statements</i>	313
6.3.5	<i>Getting Loopy</i>	315
6.3.6	<i>Arrays</i>	317
6.3.7	<i>Function Calls</i>	320
6.3.8	<i>Pseudoinstructions</i>	330

6.4	Machine Language	332
6.4.1	<i>R-Type Instructions</i>	332
6.4.2	<i>I-Type Instructions</i>	334
6.4.3	<i>S/B-Type Instructions</i>	336
6.4.4	<i>U/J-Type Instructions</i>	338
6.4.5	<i>Immediate Encodings</i>	340
6.4.6	<i>Addressing Modes</i>	341
6.4.7	<i>Interpreting Machine Language Code</i>	342
6.4.8	<i>The Power of the Stored Program</i>	343
6.5	Lights, Camera, Action: Compiling, Assembling, and Loading	344
6.5.1	<i>The Memory Map</i>	344
6.5.2	<i>Assembler Directives</i>	346
6.5.3	<i>Compiling</i>	348
6.5.4	<i>Assembling</i>	350
6.5.5	<i>Linking</i>	353
6.5.6	<i>Loading</i>	355
6.6	Odds and Ends	355
6.6.1	<i>Endianness</i>	355
6.6.2	<i>Exceptions</i>	356
6.6.3	<i>Signed and Unsigned Instructions</i>	360
6.6.4	<i>Floating-Point Instructions</i>	361
6.6.5	<i>Compressed Instructions</i>	362
6.7	Evolution of the RISC-V Architecture	363
6.7.1	<i>RISC-V Base Instruction Sets and Extensions</i>	364
6.7.2	<i>Comparison of RISC-V and MIPS Architectures</i>	365
6.7.3	<i>Comparison of RISC-V and ARM Architectures</i>	365
6.8	Another Perspective: x86 Architecture	366
6.8.1	<i>x86 Registers</i>	366
6.8.2	<i>x86 Operands</i>	367
6.8.3	<i>Status Flags</i>	369
6.8.4	<i>x86 Instructions</i>	369
6.8.5	<i>x86 Instruction Encoding</i>	371
6.8.6	<i>Other x86 Peculiarities</i>	372
6.8.7	<i>The Big Picture</i>	373
6.9	Summary	374
	Exercises	375
	Interview Questions	390
	Chapter 7 Microarchitecture	393
7.1	Introduction	393
7.1.1	<i>Architectural State and Instruction Set</i>	393
7.1.2	<i>Design Process</i>	394
7.1.3	<i>Microarchitectures</i>	396

7.2	Performance Analysis	397
7.3	Single-Cycle Processor	398
7.3.1	<i>Sample Program</i>	399
7.3.2	<i>Single-Cycle Datapath</i>	399
7.3.3	<i>Single-Cycle Control</i>	407
7.3.4	<i>More Instructions</i>	410
7.3.5	<i>Performance Analysis</i>	412
7.4	Multicycle Processor	415
7.4.1	<i>Multicycle Datapath</i>	416
7.4.2	<i>Multicycle Control</i>	422
7.4.3	<i>More Instructions</i>	432
7.4.4	<i>Performance Analysis</i>	435
7.5	Pipelined Processor	439
7.5.1	<i>Pipelined Datapath</i>	441
7.5.2	<i>Pipelined Control</i>	443
7.5.3	<i>Hazards</i>	443
7.5.4	<i>Performance Analysis</i>	454
7.6	HDL Representation	456
7.6.1	<i>Single-Cycle Processor</i>	457
7.6.2	<i>Generic Building Blocks</i>	461
7.6.3	<i>Testbench</i>	464
7.7	Advanced Microarchitecture	468
7.7.1	<i>Deep Pipelines</i>	468
7.7.2	<i>Micro-Operations</i>	469
7.7.3	<i>Branch Prediction</i>	470
7.7.4	<i>Superscalar Processors</i>	472
7.7.5	<i>Out-of-Order Processor</i>	473
7.7.6	<i>Register Renaming</i>	476
7.7.7	<i>Multithreading</i>	478
7.7.8	<i>Multiprocessors</i>	479
7.8	Real-World Perspective: Evolution of RISC-V Microarchitecture	482
7.9	Summary	486
	Exercises	488
	Interview Questions	497
Chapter 8 Memory Systems		499
8.1	Introduction	499
8.2	Memory System Performance Analysis	503
8.3	Caches	505
8.3.1	<i>What Data is Held in the Cache?</i>	505
8.3.2	<i>How is Data Found?</i>	506
8.3.3	<i>What Data is Replaced?</i>	514
8.3.4	<i>Advanced Cache Design</i>	515

8.4	Virtual Memory.....	519
8.4.1	Address Translation.....	522
8.4.2	The Page Table.....	523
8.4.3	The Translation Lookaside Buffer.....	525
8.4.4	Memory Protection.....	526
8.4.5	Replacement Policies.....	527
8.4.6	Multilevel Page Tables	527
8.5	Summary.....	530
	Epilogue.....	530
	Exercises	532
	Interview Questions	541
	Chapter 9 Embedded I/O Systems	542
9.1	Introduction.....	542
	Chapter 9 is available as an online supplement.....	542.e1
9.1	Introduction.....	542.e1
9.2	Memory-Mapped I/O.....	542.e1
9.3	Embedded I/O Systems	542.e3
9.3.1	RED-V Board.....	542.e3
9.3.2	FE310-G002 System-on-Chip	542.e5
9.3.3	General-Purpose Digital I/O	542.e5
9.3.4	Device Drivers	542.e10
9.3.5	Serial I/O.....	542.e14
9.3.6	Timers.....	542.e29
9.3.7	Analog I/O	542.e30
9.3.8	Interrupts.....	542.e39
9.4	Other Microcontroller Peripherals	542.e43
9.4.1	Character LCDs.....	542.e43
9.4.2	VGA Monitor.....	542.e45
9.4.3	Bluetooth Wireless Communication	542.e53
9.4.4	Motor Control	542.e54
9.5	Summary.....	542.e64
	Appendix A Digital System Implementation	543
A.1	Introduction.....	543
	Appendix A is available as an online supplement.....	543.e1
A.1	Introduction.....	543.e1
A.2	74xx Logic	543.e1
A.2.1	Logic Gates.....	543.e2
A.2.2	Other Functions.....	543.e2

A.3	Programmable Logic	543.e2
A.3.1	PROMs	543.e2
A.3.2	PLAs	543.e6
A.3.3	FPGAs	543.e7
A.4	Application-Specific Integrated Circuits	543.e9
A.5	Datasheets	543.e9
A.6	Logic Families	543.e15
A.7	Switches and Light-Emitting Diodes	543.e17
A.7.1	Switches	543.e17
A.7.2	LEDs	543.e18
A.8	Packaging and Assembly	543.e19
A.8.1	Packages	543.e19
A.8.2	Breadboards	543.e20
A.8.3	Printed Circuit Boards	543.e20
A.8.4	Putting It All Together	543.e23
A.9	Transmission Lines	543.e23
A.9.1	Matched Termination	543.e24
A.9.2	Open Termination	543.e26
A.9.3	Short Termination	543.e27
A.9.4	Mismatched Termination	543.e27
A.9.5	When to Use Transmission Line Models	543.e30
A.9.6	Proper Transmission Line Terminations	543.e30
A.9.7	Derivation of Z_0	543.e32
A.9.8	Derivation of the Reflection Coefficient	543.e33
A.9.9	Putting It All Together	543.e34
A.10	Economics	543.e35

Appendix B RISC-V Instruction Set Summary 544

Appendix C C Programming 545

C.1	Introduction	545
-----	--------------------	-----

Appendix C is available as an online supplement 545.e1

C.1	Introduction	545.e1
C.2	Welcome to C	545.e3
C.2.1	C Program Dissection	545.e3
C.2.2	Running a C Program	545.e4
C.3	Compilation	545.e5
C.3.1	Comments	545.e5
C.3.2	#define	545.e5
C.3.3	#include	545.e6

C.4	Variables	545.e7
C.4.1	<i>Primitive Data Types</i>	545.e8
C.4.2	<i>Global and Local Variables</i>	545.e9
C.4.3	<i>Initializing Variables</i>	545.e11
C.5	Operators	545.e11
C.6	Function Calls	545.e15
C.7	Control-Flow Statements	545.e16
C.7.1	<i>Conditional Statements</i>	545.e17
C.7.2	<i>Loops</i>	545.e19
C.8	More Data Types.....	545.e21
C.8.1	<i>Pointers</i>	545.e21
C.8.2	<i>Arrays</i>	545.e23
C.8.3	<i>Characters</i>	545.e27
C.8.4	<i>Strings</i>	545.e27
C.8.5	<i>Structures</i>	545.e29
C.8.6	<i>typedef</i>	545.e31
C.8.7	<i>Dynamic Memory Allocation</i>	545.e32
C.8.8	<i>Linked Lists</i>	545.e33
C.9	Standard Libraries.....	545.e35
C.9.1	<i>stdio</i>	545.e35
C.9.2	<i>stdlib</i>	545.e40
C.9.3	<i>math</i>	545.e42
C.9.4	<i>string</i>	545.e42
C.10	Compiler and Command Line Options.....	545.e43
C.10.1	<i>Compiling Multiple C Source Files</i>	545.e43
C.10.2	<i>Compiler Options</i>	545.e43
C.10.3	<i>Command Line Arguments</i>	545.e44
C.11	Common Mistakes	545.e44
	Further Reading	547
	Index	549

Preface

This book is unique in its treatment in that it presents digital logic design from the perspective of computer architecture, starting at the beginning with 1's and 0's and leading through to the design of a microprocessor.

We believe that building a microprocessor is a special rite of passage for engineering and computer science students. The inner workings of a processor seem almost magical to the uninitiated yet prove to be straightforward when carefully explained. Digital design in and of itself is a powerful and exciting subject. Assembly language programming unveils the inner language spoken by the processor. Microarchitecture is the link that brings it all together.

The first two versions of this increasingly popular text cover the MIPS and ARM architectures. As one of the original Reduced Instruction Set Computing architectures, MIPS is clean and exceptionally easy to understand and build. MIPS remains an important architecture, as it has inspired many of the subsequent architectures, including RISC-V. The ARM architecture has exploded in popularity over the past several decades because of its efficiency and rich ecosystem. More than 50 billion ARM processors have been shipped, and more than 75% of humans on the planet use products with ARM processors.

Over the past decade, RISC-V has emerged as an increasingly important architecture, both pedagogically and commercially. As the first widely used open-source computer architecture, RISC-V offers the simplicity of MIPS with the flexibility and features of modern processors.

Pedagogically, the learning objectives of the MIPS, ARM, and RISC-V editions are identical. The RISC-V architecture has a number of features, including extendibility and compressed instructions, that contribute to its efficiency but add a small amount of complexity. The three microarchitectures are also similar, with MIPS and RISC-V architectures sharing many similarities. We expect to offer MIPS, ARM, and RISC-V editions as long as the market demands.

FEATURES

Side-by-Side Coverage of SystemVerilog and VHDL

Hardware description languages (HDLs) are at the center of modern digital design practices. Unfortunately, designers are evenly split between the two dominant languages, SystemVerilog and VHDL. This book introduces HDLs in [Chapter 4](#) as soon as combinational and sequential logic design has been covered. HDLs are then used in [Chapters 5](#) and [7](#) to design larger building blocks and entire processors. Nevertheless, [Chapter 4](#) can be skipped and the later chapters are still accessible for courses that choose not to cover HDLs.

This book is unique in its side-by-side presentation of SystemVerilog and VHDL, enabling the reader to learn the two languages. [Chapter 4](#) describes principles that apply to both HDLs, and then provides language-specific syntax and examples in adjacent columns. This side-by-side treatment makes it easy for an instructor to choose either HDL and for the reader to transition from one to the other, either in a class or in professional practice.

RISC-V Architecture and Microarchitecture

[Chapters 6](#) and [7](#) offer in-depth coverage of the RISC-V architecture and microarchitecture. RISC-V is an ideal architecture because it is a real architecture shipped in an increasing number of commercial products, yet it is streamlined and easy to learn. Moreover, because of its popularity in the commercial and hobbyist worlds, simulation and development tools exist for the RISC-V architecture.

Real-World Perspectives

In addition to the real-world perspective in discussing the RISC-V architecture, [Chapter 6](#) illustrates the architecture of Intel x86 processors to offer another perspective. [Chapter 9](#) (available as an online supplement) also describes peripherals in the context of SparkFun's RED-V RedBoard, a popular development board that centers on SiFive's Freedom E310 RISC-V processor. These real-world perspective chapters show how the concepts in the chapters relate to the chips found in many PCs and consumer electronics.

Accessible Overview of Advanced Microarchitecture

[Chapter 7](#) includes an overview of modern high-performance microarchitectural features, including branch prediction, superscalar, and out-of-order operation, multithreading, and multicore processors. The

treatment is accessible to a student in a first course and shows how the microarchitectures in the book can be extended to modern processors.

End-of-Chapter Exercises and Interview Questions

The best way to learn digital design is to do it. Each chapter ends with numerous exercises to practice the material. The exercises are followed by a set of interview questions that our industrial colleagues have asked students who are applying for work in the field. These questions provide a helpful glimpse into the types of problems that job applicants will typically encounter during the interview process. Exercise solutions are available via the book's companion and instructor websites.

ONLINE SUPPLEMENTS

Supplementary materials are available online at ddcabook.com or the publisher's website: <https://www.elsevier.com/books-and-journals/book-companion/9780128200643>. These companion sites (accessible to all readers) include the following:

- ▶ Links to video lectures
- ▶ Solutions to odd-numbered exercises
- ▶ Figures from the text in PDF and PPTX formats
- ▶ Links to professional-strength computer-aided design (CAD) tools from Intel[®]
- ▶ Instructions on how to use PlatformIO (an extension of Visual Studio Code) to compile, assemble, and simulate C and assembly code for RISC-V processors
- ▶ Hardware description language (HDL) code for the RISC-V processor
- ▶ Intel's Quartus helpful hints
- ▶ Lecture slides in PowerPoint (PPTX) format
- ▶ Sample course and laboratory materials
- ▶ List of errata

The instructor site (accessible to instructors who register at <https://inspectioncopy.elsevier.com>) includes the following:

- ▶ Solutions to all exercises
- ▶ Laboratory solutions

EdX MOOC

This book also has a companion Massive Open Online Course (MOOC) through EdX. The course includes video lectures, interactive practice

problems, and interactive problem sets and labs. The MOOC is divided into two parts: Digital Design (ENGR 85A) and Computer Architecture (ENGR85B) offered by HarveyMuddX (on EdX, search for “Digital Design HarveyMuddX” and “Computer Architecture HarveyMuddX”). EdX does not charge for access to the videos but does charge for the interactive exercises and certificate. EdX offers discounts for students with financial need.

HOW TO USE THE SOFTWARE TOOLS IN A COURSE

Intel’s Quartus Software

The Quartus software, either Web or Lite Edition, is a free version of Intel’s professional-strength Quartus™ FPGA design tools. It allows students to enter their digital designs in schematic or using either the SystemVerilog or the VHDL hardware description language (HDL). After entering the design, students can simulate their circuits using the ModelSim™-Intel FPGA Edition or Starter Edition, which is available with Intel’s Quartus software. Quartus also includes a built-in logic synthesis tool that supports both SystemVerilog and VHDL.

The difference between the Web or Lite Edition and the Pro Edition is that the Web or Lite Edition supports a subset of the most common Altera FPGAs. The free versions of ModelSim degrade performance for simulations with more than 10,000 lines of HDL, whereas the professional version of ModelSim does not.

PlatformIO

PlatformIO, which is an extension of Visual Studio Code, serves as a software development kit (SDK) for RISC-V. With the explosion of SDKs for each new platform, PlatformIO has streamlined the process of programming and using various processors by providing a unified interface for a large number of platforms and devices. It is available as a free download and can be used with SparkFun’s RED-V RedBoard, as described in the labs provided on the companion website. PlatformIO provides access to a commercial RISC-V compiler and allows students to write both C and assembly programs, compile them, and then run and debug them on SparkFun’s RED-V RedBoard (see [Chapter 9](#) and the accompanying labs).

Venus RISC-V Assembly Simulator

The Venus Simulator (available at: <https://www.kvakil.me/venus/>) is a web-based RISC-V assembly simulator. Programs are written (or copy/pasted) in the Editor tab and then simulated and run in the Simulator tab. Registers and memory contents can be viewed as the program runs.

LABS

The companion site includes links to a series of labs that cover topics from digital design through computer architecture. The labs teach students how to use the Quartus tools to enter, simulate, synthesize, and implement their designs. The labs also include topics on C and assembly language programming using PlatformIO and SparkFun's RED-V RedBoard.

After synthesis, students can implement their designs using the Altera DE2, DE2-115, DE0, or other FPGA board. The labs are written to target the DE2 or DE-115 boards. These powerful and competitively priced boards are available from de2-115.terasic.com. The board contains an FPGA that can be programmed to implement student designs. We provide labs that describe how to implement a selection of designs on the DE2-115 board using the Quartus software.

To run the labs, students will need to download and install Intel's Quartus Web or Lite Edition and Visual Studio Code with the PlatformIO extension. Instructors may also choose to install the tools on lab machines. The labs include instructions on how to implement the projects on the DE2/DE2-115 board. The implementation step may be skipped, but we have found it of great value. We have tested the labs on Windows, but the tools are also available for Linux.

RVfpga

RISC-V FPGA, also referred to as RVfpga, is a free two-course sequence that can be completed after learning the material in this book. The first course shows how to target a commercial RISC-V core to an FPGA, program it using RISC-V assembly or C, add peripherals to it, and analyze and modify the core and memory system, including adding instructions to the core. This course uses the open-source SweRVolf system-on-chip (SoC) (<https://github.com/chipsalliance/Cores-SweRVolf>), which is based on Western Digital's open-source commercial SweRV EH1 core (<https://www.westerndigital.com/company/innovations/risc-v>). The course also shows how to use Verilator, an open-source HDL simulator, and Western Digital's Whisper, an open-source RISC-V instruction set simulator (ISS). RVfpga-SoC, the second course, shows how to build an SoC based on SweRVolf using building blocks such as the SweRV EH1 core, interconnect, and memories. The course then guides the user in loading and running the Zephyr operating system on the RISC-V SoC. All necessary software and system source code (Verilog/SystemVerilog files) are free, and the courses may be completed in simulation, so no hardware is required. RVfpga materials are freely available with registration from the Imagination Technologies University Programme: <https://university.imgtec.com/rvfpga/>.

BUGS

As all experienced programmers know, any program of significant complexity undoubtedly contains bugs. So, too, do books. We have taken great care to find and squash the bugs in this book. However, some errors undoubtedly do remain. We will maintain a list of errata on the book's webpage.

Please send your bug reports to ddcabugs@gmail.com. The first person to report a substantive bug with a fix that we use in a future printing will be rewarded with a \$1 bounty!

ACKNOWLEDGMENTS

We appreciate the hard work of Steve Merken, Nate McFadden, Ruby Gammell, Andrae Akeh, Manikandan Chandrasekaran, and the rest of the team at Morgan Kaufmann who made this book happen. We love the art of Duane Bibby, whose cartoons enliven the chapters.

We thank Matthew Watkins, who contributed the section on Heterogeneous Multiprocessors in [Chapter 7](#), and Josh Brake, who contributed to [Chapter 9](#) on Embedded I/O Systems. We greatly appreciate the work of Mateo Markovic and Geordie Ryder, who reviewed the book and contributed to the exercise solutions. Numerous other reviewers also substantially improved the book. They include Daniel Chaver Martinez, Roy Kravitz, Zvonimir Bandic, Giuseppe Di Luna, Steffen Paul, Ravi Mittal, Jennifer Winikus, Hesham Omran, Angel Solis, Reiner Dizon, and Olof Kindgren. We also appreciate the students in our courses at Harvey Mudd College and UNLV who have given us helpful feedback on drafts of this textbook. And last, but not least, we both thank our families for their love and support.

About the Authors

Sarah L. Harris is an Associate Professor of Electrical and Computer Engineering at the University of Nevada, Las Vegas. She received her Ph.D. and M.S. in Electrical Engineering from Stanford University. Sarah has also worked with Hewlett-Packard, the San Diego Supercomputer Center, and NVIDIA. Sarah loves teaching, exploring and developing new technologies, traveling, playing the guitar and various other instruments, and spending time with her family. Her recent research includes designing bio-inspired prosthetics and implementing machine-learning algorithms in hardware.

David Harris is the Harvey S. Mudd Professor of Engineering Design and Associate Department Chair at Harvey Mudd College. He received his Ph.D. in electrical engineering from Stanford University and his M.Eng. in electrical engineering and computer science from MIT. Before attending Stanford, he worked at Intel as a logic and circuit designer on the Itanium and Pentium II processors. Since then, he has consulted at Broadcom, Sun Microsystems, Hewlett-Packard, Evans & Sutherland, and other design companies. David's passions include teaching, building chips and airplanes, and exploring the outdoors. When he is not at work, he can usually be found hiking, flying, and spending time with his family. David holds about a dozen patents and is the author of three other textbooks on chip design, as well as seven guidebooks to the Southern California mountains.

